

# 产品需求文档 (PRD) - IndusAI Pulse

版本： V1.0

定位： 专为工业互联网 PM 打造的“AI+制造”情报聚合平台。

核心目标： 自动监测 GitHub AI 项目，通过工业场景算法过滤，提供可落地的降本增效技术情报。

## 一、核心功能细化

### 1.1 前端展示层 (UX/UI)

采用极简 Linear/Vercel 极客风（暗黑模式优先）。

| 模块    | 功能项      | 需求细化说明                                            | 工业 PM 定制点                       |
|-------|----------|---------------------------------------------------|---------------------------------|
| 全局切换  | 双 Tab 导航 | 顶部 📅 每日早报 vs 🚀 Trending 榜单 分段控制器。                 | 保持高频阅读与低频调研的解耦。                 |
| Tab 1 | 资讯卡片流    | 卡片包含：中文标题、一句话摘要、领域标签（如：缺陷检测）、GitHub 链接。           | 新增：“降本增效”潜能标识（💰 / ⚡）。           |
|       | 手风琴交互    | 点击卡片向下平滑展开：展示“项目亮点”、“工业适用场景”、“预估接入难度”。            | 拒绝通用描述，直接输出“该算法可用于皮带撕裂检测”等业务描述。 |
|       | 日历导航     | 侧边或顶部悬浮月历，点击日期跳转。                                 | 方便复盘过去一周的行业突发技术。                |
| Tab 2 | 数据榜单     | 24H/7D 维度，Top 20 排行。展示新增 Star 数、总 Star 数、开发者、主语言。 | 新增：“工业相关度分数”。                   |

### 1.2 工业化过滤逻辑 (核心业务逻辑)

这是将通用 GitHub 数据转化为“工业 AI 知识”的关键步。

#### 后端 AI 过滤指令 (Prompt Injection):

在调用大模型处理 GitHub Readme 时，强制执行以下过滤规则：

- 关键词加权：** 包含 Time Series (时间序列)、Anomaly Detection (异常检测)、Predictive Maintenance (预测性维护)、CV-Inspection (视觉质检)、PLC、Optimization (运筹优化) 的项目优先。
- 黑名单过滤：** 自动剔除 Roleplay (角色扮演)、Anime (动漫生成)、Art Generation (纯艺术类)、Writing Assistant (写作助理) 等项目。
- 工业落地评估：** AI 需回答：该技术是否能在离线环境下运行？是否支持边缘端设备（如 Jetson, Raspberry Pi）？

## 二、后端 workflow 与数据架构

### 2.1 自动化引擎流程 (无人值守)

- 抓取 (Trigger):** 每晨 06:00 (UTC+8) 触发 GitHub GraphQL API 抓取。
- 清洗 (Pre-process):** 提取 Readme.md 前 2000 字符。
- AI 摘要 (Transformation):**
  - 输入:** 原始 Readme。
  - 输出:** 严格 JSON 格式。

codeJSON

```
{
  "title": "中文简短标题",
  "summary": "一句话介绍",
  "industrial_value": "描述该项目如何帮助工厂降本增效",
  "tags": ["视觉质检", "边缘计算"],
  "difficulty": "高/中/低",
  "is_industrial_relevant": true/false
}
```

- 存储与上架 (Load):** 如果 is\_industrial\_relevant 为 true 且格式正确，写入 Supabase，否则丢弃。

## 2.2 容错与“防翻车”机制

- JSON 验证器:** 后端设置 Schema 校验，凡是不符合格式的 AI 返回结果，重试一次，失败则记录 Log 并丢弃。
- 空状态处理:** 若当日无工业相关高质量项目，早报显示：“今日工业 AI 领域相对平静，建议回顾上周 Trending”。

# 三、功能清单

## 3.1 🧩 前端功能（用户展示层）

整体页面采用**双 Tab 结构**切换，保持界面极致清爽。

### 3.1.1 页面框架

| 模块   | 功能名称   | 功能描述                                                          |
|------|--------|---------------------------------------------------------------|
| 全局导航 | Tab 切换 | 顶部或左侧常驻两个 Tab：<br>Tab 1: 📅 每日早报 (默认)<br>Tab 2: 🚀 Trending 飙升榜 |

### 3.1.2 Tab 1: 📅 每日早报（带有日历功能的资讯流）

主打**沉浸式阅读与历史回顾**。

| 模块   | 功能名称      | 功能描述                                                    |
|------|-----------|---------------------------------------------------------|
| 今日核心 | 每日 AI 资讯流 | 按天展示精选的 AI 资讯卡片。<br>展示：AI生成的中文标题、一句话摘要、标签、直达 GitHub 按钮。 |

| 模块   | 功能名称   | 功能描述                                           |
|------|--------|------------------------------------------------|
| 阅读体验 | 手风琴式展开 | 点击卡片，直接在当前页面向下展开详细的“AI 深度总结（亮点、适用场景）”，不破坏阅读心流。 |
| 历史归档 | 日历导航器  | 核心交互组件（月历形态）。有数据的日期带标记，点击任意日期自动加载当天的资讯流。       |
|      | 快捷翻页   | 资讯流底部提供 ← 上一天 / 下一天 → 按钮。                      |

### 3.1.3 Tab 2：🚀 Trending 飙升榜（数据驱动潜力股发现）

主打客观数据展示，满足开发者“找新轮子”的硬核需求。

| 模块     | 功能名称   | 功能描述                                                                                                                                                                                                  |
|--------|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 时间维度筛选 | 榜单周期切换 | 在榜单顶部提供两个二级按钮：过去 24H / 过去 7 天。点击无刷新切换榜单数据。                                                                                                                                                            |
| 榜单呈现   | 排行榜列表  | 以 Top 1 到 Top 20 的列表形式展示。<br>每行包含：<br>1. 排名序号（前三名用特殊颜色或奖牌高亮）；<br>2. 项目名称 ( owner/repo ) ；<br>3. 原生简介（调用 GitHub 原生 Description，可借助 AI 翻译成一句话中文）；<br>4. 核心指标 🔥：高亮展示新增 Star 数（如：🔥 +540 Stars ）和总 Star 数。 |
| 快捷操作   | 直达链接   | 点击项目所在行，直接在新标签页跳转至对应 GitHub 仓库。                                                                                                                                                                       |

### 3.2 ⚙️ 后端工作流（100% 全自动化无人值守引擎）

因为有了管理后台，后端的“防翻车机制”和“数据准确性”成为了重点。

| 模块               | 功能名称              | 功能描述                                                                                                                                                                                    |
|------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tab 1 数据源 (内容抓取) | AI 精选自动化引擎        | 1. 抓取：定时拉取 GitHub 带有 AI/LLM/Agent 等标签的新晋热门仓库及 Awesome 榜单更新。<br>2. AI 过滤与生成：交由大模型处理，并强制设定 Prompt：“如果该仓库没有实质代码或没有 README，返回 null”。<br>3. 自动上架：大模型返回合法 JSON 后，自动写入数据库并绑定当日日期 publish_date。 |
| Tab 2 数据源 (趋势监控) | Trending 爬虫引擎     | 1. 抓取 GitHub 官方 trending 页面（过滤语言为 Python/C++ 或含 AI 关键词的项目），分别抓取 ?since=daily (24H) 和 ?since=weekly (7 天) 的数据。<br>2. 提取出 repo_name 和当期新增的 Stars 数量。<br>3. 每日更新这部分数据并覆盖展示。                |
| 容错机制             | 内容自动丢弃 (Fallback) | 既然没有人工审核，如果某一条数据大模型翻译失败，或者输出的 JSON 格式错乱，后端脚本直接 Try-Catch 丢弃该条数据，宁缺毋滥，保证前端页面不崩。                                                                                                          |

### 3.3 💡 原型/UI 构思建议

这套结构非常适合做成类似 **Vercel** 或 **Linear** 的那种极简暗黑风/极客风界面。

- **首页布局：**
  - 最顶部是产品的 Logo 和一句 Slogan（比如："Explore the Frontier of AI Daily"）。
  - 紧接着是两个大大的分段控制器（Segmented Control）：[ 📅 每日早报 | 🚀 Trending 榜单 ]
- **当停留在“早报”时：**左侧展示一个精致的小日历，右侧是当日的资讯卡片流。
- **当切换到“Trending”时：**日历隐藏，页面变成一个干净的排行榜。右侧显示绿色的 +XXX Stars 极其抓人眼球。

开发用 **Next.js** 搭建前后端，数据库用轻量级的 **Supabase**。利用 GitHub Actions 每天跑一次 Python 脚本去抓数据 + 调大模型 API 写入数据库。整个产品全自动运转，用户只需作为“第一个真实用户”去天天看它即可！

## 四、技术栈实现建议 (MVP 版本)

- **前端框架：**Next.js (支持 SEO，虽然是自用，但加载速度快)。
- **UI 组件库：**Shadcn/ui (符合你要求的 Linear 极简风)。
- **后端/数据库：**Supabase (处理数据存储与 API)。
- **爬虫与 AI 调度：**GitHub Actions (每天定时运行 Python 脚本，0 成本)。
- **大模型接口：**DeepSeek 或 GPT-4o-mini (成本极低，逻辑极强)。

## 五、基础的 GitHub API 抓取模版（python）

### 5.1 🛠️ 准备工作

1. **GitHub Token:** 在 GitHub 设置中申请一个 Personal Access Token (Fine-grained)，用于提高 API 调用频率限制。
2. **AI API Key:** OpenAI、DeepSeek 或其他大模型厂商的 API Key。

### 5.2 🐍 Python 脚本模版：IndusAI\_Pulse\_Engine.py

codePython

```
import requests
import json
import time
from datetime import datetime, timedelta
from openai import OpenAI

# --- 配置区 ---
GITHUB_TOKEN = "你的_GITHUB_TOKEN"
AI_API_KEY = "你的_AI_API_KEY"
```

```

AI_BASE_URL = "https://api.deepseek.com" # 或者 OpenAI 的 URL
GITHUB_API_URL = "https://api.github.com/search/repositories"

# 实例化 AI 客户端
client = OpenAI(api_key=AI_API_KEY, base_url=AI_BASE_URL)

# 你之前定义的 Prompt
SYSTEM_PROMPT = """
你是一位资深的“工业 AI 解决方案专家”和“B 端产品经理”。
你的任务是从 GitHub 项目中筛选出能为智能制造带来“降本增效”价值的技术。
（此处粘贴完整的 Prompt 逻辑内容...）
"""

def get_trending_ai_repos():
    """从 GitHub 获取过去 24 小时内最热门的 AI 相关仓库"""
    yesterday = (datetime.now() - timedelta(days=1)).strftime('%Y-%m-%d')
    # 构造查询: AI/LLM 相关, 创建时间在昨天之后, 按 Star 数排序
    query = f"topic:ai OR topic:llm OR topic:machine-learning created:>{yesterday}"
    headers = {"Authorization": f"token {GITHUB_TOKEN}"}
    params = {
        "q": query,
        "sort": "stars",
        "order": "desc",
        "per_page": 10 # 每天精选前 10 个进行深度分析
    }

    response = requests.get(GITHUB_API_URL, headers=headers, params=params)
    if response.status_code == 200:
        return response.json().get('items', [])
    else:
        print(f"Error fetching GitHub: {response.status_code}")
        return []

def get_repo_readme(full_name):
    """获取指定仓库的 README 内容"""
    url = f"https://api.github.com/repos/{full_name}/readme"
    headers = {
        "Authorization": f"token {GITHUB_TOKEN}",
        "Accept": "application/vnd.github.raw"
    }
    response = requests.get(url, headers=headers)
    if response.status_code == 200:
        # 截取前 3000 字符, 防止 Token 溢出
        return response.text[:3000]
    return ""

def analyze_repo_with_ai(readme_content):
    """调用大模型进行工业价值分析"""
    try:
        response = client.chat.completions.create(
            model="deepseek-chat", # 或 gpt-4o-mini
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},

```

`{"role": "user", "content": f"请分析以下 README 内容:"`

```
\n\n{readme_content}"})
    ],
    response_format={"type": "json_object"} # 强制输出 JSON
)
result = json.loads(response.choices[0].message.content)
return result
except Exception as e:
    print(f"AI Analysis Error: {e}")
    return None

def main():
    print(f"--- 启动任务: {datetime.now()} ---")
    repos = get_trending_ai_repos()
    daily_report = []

    for repo in repos:
        name = repo['full_name']
        print(f"正在分析仓库: {name}...")

        readme = get_repo_readme(name)
        if not readme:
            continue

        analysis = analyze_repo_with_ai(readme)

        # 只有当 AI 认为与工业相关时才保留
        if analysis and analysis.get('is_industrial_relevant'):
            # 补充仓库基本信息
            analysis['html_url'] = repo['html_url']
            analysis['stars_count'] = repo['stargazers_count']
            daily_report.append(analysis)
            print(f"✅ 已收录: {analysis['zh_title']}")
        else:
            print(f"❌ 过滤非工业相关项目: {name}")

        # 频率控制, 避免触发 API 限制
        time.sleep(1)

    # 将结果保存为 JSON 文件, 供前端展示
    output_file = f"report_{datetime.now().strftime('%Y%m%d')}.json"
    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(daily_report, f, ensure_ascii=False, indent=2)

    print(f"--- 任务完成, 生成报告: {output_file} ---")

if __name__ == "__main__":
    main()
```

## 5.3 💡 脚本关键逻辑说明:

- 动态查询策略 (Query Strategy):**
  - 脚本使用了 GitHub Search API 搜索 topic:ai 且 created:>昨日 的项目。这比抓取官方 Trending 页面更稳定，因为 API 返回的是结构化数据。
- README 截断:**
  - readme[:3000]: 由于工业项目的 README 往往包含大量的安装步骤、引用论文，这些对 AI 判断业务价值用处不大。截取前 3000 字通常已覆盖“Introduction”和“Key Features”，能有效节省 Token 成本。
- JSON 强校验:**
  - 脚本利用了部分模型（如 GPT-4, DeepSeek）支持的 response\_format={"type": "json\_object"} 功能，确保后端拿到的数据可以直接被前端解析，不会因为多了一句“Here is your summary”而崩掉。
- 业务字段补全:**
  - 除了 AI 生成的内容，脚本还抓取了 stargazers\_count（Star 数）和 html\_url（直达链接），这些是你在 Tab 2 (Trending) 中需要的原始数据。

## 六、🤖 工业 AI 过滤与总结提示词 (Prompt)

### Role:

你是一位资深的“工业 AI 解决方案专家”和“B 端产品经理”。你擅长从复杂的 GitHub 开源项目中，筛选出能为智能制造、数字化工厂带来“降本增效”价值的技术。

### Task:

请分析以下 GitHub 仓库的 README 内容，判断其是否具有工业应用潜力。如果符合，请按要求输出结构化的中文摘要；如果不符合，请直接返回 null。

## 1. 筛选准则（过滤逻辑）

【必须保留】 符合以下任一特征的项目：

- 工业视觉：** 表面缺陷检测、视觉测量、OCR 零件识别、工装佩戴监控。
- 时间序列：** 预测性维护（PdM）、设备余寿预测（RUL）、能源负荷预测、异常检测。
- 运筹优化：** 车间排产（APS）、物流路径规划、库存周转率优化、供应链优化。
- 工业大模型：** 设备维修手册问答、PLC 代码辅助生成、现场作业安全助手。
- 边缘 AI：** 模型量化/剪枝（适配 Jetson/树莓派）、工业协议接入（OPC-UA/Modbus）相关的 AI 框架。

【必须剔除】 符合以下特征的项目（返回 null）：

- 纯艺术创作类（文生图、动漫风格转换、换脸）。
- 娱乐角色扮演（Roleplay）、小说写作辅助。
- 纯科研学术论文代码库（无实际应用场景说明）。
- 仅有 Prompt 集合或文章链接，无实质性代码。

## 2. 处理流程

- 识别：** 判断项目所属的工业 AI 子领域。
- 分析：** 提取核心功能，评估其对工厂“降本”或“增效”的具体贡献。
- 转化：** 将深奥的技术术语转化为 PM 能听懂的业务语言。

### 3. 输出格式 (严格 JSON)

codeJSON

```
{
  "is_industrial_relevant": true,
  "project_name": "项目名称",
  "category": "所属工业子领域",
  "zh_title": "中文直观标题 (控制在 15 字以内)",
  "one_sentence_summary": "一句话介绍项目核心技术",
  "business_value": {
    "cost_reduction": "如何降本 (例如: 减少人工巡检频次, 降低 20% 原材料损耗)",
    "efficiency_boost": "如何增效 (例如: 提升质检检出率, 缩短 10% 排产时间)"
  },
  "industrial_scenarios": ["场景 A", "场景 B"],
  "tech_stack": "核心框架/算法",
  "deployment_difficulty": "低/中/高 (基于 README 评估)"
}
```

### 4. 输入数据 (README)

{{repo\_readme\_content}}

### 5. 如何在你的自动化流中使用这个 Prompt?

- 大模型选择：** 建议使用 **GPT-4o** 或 **DeepSeek-V3**。因为工业术语较多，这类逻辑性强的模型表现更好。
- 前置清洗：** 在把 README 发给 AI 之前，建议先用脚本截取 README 的前 **2000-3000** 个字符。通常仓库的开头和“Features”部分已经包含了足够判断的信息，这样可以节省 Token 成本。
- 零成本尝试：**
  - 你可以先拿几个 GitHub 链接（比如 yolov8、Time-Series-Library、或者某个 Stable Diffusion 插件）手动喂给这个 Prompt，看看返回结果是否符合你的心意。
- 业务标签库：**
  - 如果你发现 AI 输出的 category 太乱，你可以给它一个固定的 Enum 列表，例如：["视觉质检", "预测性维护", "生产调度", "工业机器人", "边缘计算", "节能减排"]。

## 七、UI布局规划

### 1. 全局导航 (Global Header)



- **左侧：** IndusAI Pulse Logo + 一个微小的状态灯 (绿色表示今日已更新)。
  - **中间：** 分段切换器 (Segmented Control)。
    - [ 📅 Daily Digest | 🚀 Trending ] (采用胶囊形状，选中项有暗灰色背景)。
  - **右侧：** 搜索框 (快捷键 /) + 个人设置/关于。
- 

## 2. Tab 1: 📅 每日早报 (双栏布局)

这是你的核心阅读区，建议采用 **1:3** 的非对称比例。

### 2.1 左侧栏 (25%)：日历控制台 (Sticky Sidebar)

- **月历组件：** 极简风格。
  - 有数据的日期下方有一个绿色小圆点。
  - 今日日期用实心方块高亮。
- **分类快捷过滤：**
  - All / 视觉质检 / 预测性维护 / 产线优化 / 工业LLM。
  - 点击可快速过滤右侧列表。

### 2.2 右侧栏 (75%)：资讯流卡片 (Feed)

- **卡片设计 (Collapsed State - 折叠态)：**
    - **Header：** 左侧是 [领域标签] (如“缺陷检测”)，右侧是 Star数 (如 🌟 1.2k)。
    - **Title：** 20px 粗体，中文翻译标题 (如：“基于小样本学习的工业零件表面划痕识别”)。
    - **Summary：** 16px 灰色文字，展示 AI 生成的一句话摘要。
    - **Footer：** 显示 Github链接按钮 + 更新时间 + 业务价值预警器 (💰 降本 / ⚡ 增效 的图标)。
  - **手风琴展开态 (Expanded State)：**
    - **背景：** 展开部分采用微弱的渐变色或深蓝色底色。
    - **业务解读区 (Business Value Grid)：** 左右分栏。
      - **左：** [ 💰 降本潜力 ] 红色减号图标，列出如“减少人工标注 80%”、“降低算力成本”。
      - **右：** [ ⚡ 增效潜力 ] 绿色加号图标，列出如“提升检出率 15%”、“秒级响应”。
    - **技术栈：** 一行小字标签 (Python, PyTorch, TensorRT)。
    - **适用场景：** 几个关键词 (如：3C电子、汽车喷涂、光伏板检测)。
- 

## 3. Tab 2: 🚀 Trending 飙升榜 (单栏长列表)

追求直观、快速扫描。

- **顶部控制栏：** [ Last 24H | Last 7 Days ] 切换。
- **排行榜列表：**
  - **Row 结构：**
    1. **排名：** 01-20 数字，前三名加金/银/铜色边框。
    2. **Repo 路径：** owner / repo\_name (点击直达)。

3. **工业相关度：** 一个类似电池电量的进度条，显示 AI 计算的该项目与“工业互联网”的契合度。

4. **AI 翻译：** 原生描述的中文一句话直译。

5. **核心增长 (The "Heat" column)：**

- 绿色高亮显示 +540 Stars。
- 总 Star 数。
- 悬浮效果：** 鼠标悬停在某一行时，该行背景微亮，并显示一个“添加到收藏”的按钮。

## 4. 视觉风格指南 (Visual Spec)

| 元素   | 建议方案                                                        |
|------|-------------------------------------------------------------|
| 背景色  | <span>#09090b</span> (纯黑偏灰，类似 Vercel)                       |
| 卡片颜色 | <span>#18181b</span> (深灰色，带 1px 的 <span>#27272a</span> 边框)  |
| 文字颜色 | 标题 <span>#fafafa</span> (纯白) / 摘要 <span>#a1a1aa</span> (浅灰) |
| 强调色  | <span>#3b82f6</span> (科技蓝) / <span>#10b981</span> (工业绿)     |
| 字体   | Inter (无衬线) 或 JetBrains Mono (等宽字体，增加编程感)                   |
| 间距   | 采用 4px 为倍数的系统 (Gap 4, 8, 16, 24)                            |

## 5. 交互细节建议

- 加载态 (Skeleton Screen)：** 当你点击日历切换日期时，右侧卡片显示“骨架屏”，避免白屏焦虑。
- 无感刷新：** 切换 24H/7D 榜单时，使用无刷新请求，数字带有向上滚动的动画效果，体现“数据动态”。
- 响应式：**
  - 移动端：** 隐藏左侧日历，改为顶部横向滑动的日期条。
  - 卡片：** 展开态在移动端垂直排列。